01 / Step 6: build the Windows release

Objective

Create an installable Windows worker app that includes its Python runtime/dependencies and opens without a source checkout or repeated PowerShell startup. Owners prepare approved workloads, pair, approve, pause and stop through the interface. Docker/WSL remains an external supported prerequisite; do not silently install it or change OS/security settings.

Checkpoint - 10 October 2026

Steps 1-4 passed local acceptance. Step 5 offline distribution is implemented: 84 worker tests, 33 backend tests with zero skips, and 32 workload tests passed. A local pinned Docker job processed ten photos as two tasks with exact results. The user reports separate-laptop preflight passed; the approved job/owner/result record on that laptop is still pending. Step 6 has not been implemented or tested.

Source handoff first

Repository: https://github.com/dkush2024-dot/E-waste-computer-grid. Current local base: a14f604. Step 5 changes remain local/uncommitted at this snapshot. Obtain the full current source or a reviewed later commit that contains worker/runtime_release.py and releases/current.json. The worker-only source ZIP is useful for testing, but is not the complete development repository or an installer.

Scope and proposed approach

Recommended starting design: a PyInstaller one-folder bundle with separate GUI and agent entry points, then a per-user Windows installer. This is a proposal, not existing code. One-folder packaging keeps resource/helper paths easy to inspect during development. Select the installer compiler after checking tool availability and current licensing. Avoid a Windows service, auto-updater or new queue technology without a concrete requirement.

Dependencies and permission

Complete or explicitly retain the Step 5 physical gate; independent packaging development can proceed meanwhile. Freeze the reviewed source/release before acceptance. This handoff authorizes no desktop control, publishing, messages, credential access or system-wide installation on another person's machine. Agents follow their own user's scope and approval controls. This document-generation task changed documentation only.

Definition of done

Versioned GUI/agent bundle, installer, shortcuts, visible tray/exit behavior, opt-in login startup, graceful update/uninstall, sanitized diagnostics, exact image-release identity, and real clean-machine evidence. A developer-machine build is not proof that a computer without Python can run it. Preserve all existing consent, protected-state, resource and lease guarantees.

02 / Understand the existing code

Computation and launch path

worker/desktop.py is the Tkinter owner interface. desktop_runtime.py runs preflight and launches the existing Python agent as a child. agent.py performs heartbeat, task claims, cancellation and result reporting. task_executor.py stages inputs and runs approved restricted Docker containers. Keep computation here; do not move it into the tray or GUI thread.

State and consent

pairing_client.py handles expiring invitation/decision, identity recovery, grid binding and pause. state_store.py provides per-user Windows DPAPI protection plus atomic file updates. Credentials/settings remain under the per-grid %LOCALAPPDATA%\EWasteGrid state tree. An existing approval keeps its limits and identity; pause survives restart. Revoked credentials must not silently re-enroll.

Offline release

runtime_release.py verifies the trusted manifest/archive, checks Linux/amd64 image IDs, imports explicitly and validates cache. config.py carries release selection into the child. The model/labels live inside the inference image; the host desktop does not need a second ONNX model copy. Bundle the trusted current.json, preserve its fingerprint check, and transfer the ignored images.tar separately for this release.

Three actual gaps

First, DesktopRuntime currently invokes [sys.executable, agent.py]. In a frozen app, sys.executable identifies the app executable rather than a standalone Python interpreter [1]. Second, module-relative resources and installed writable paths need explicit handling. Third, no packaged tray/startup/installer/upgrade/uninstall lifecycle exists. These gaps cannot be solved by merely renaming a ZIP to an installer.

Files to inspect before edits

Read worker/desktop_runtime.py, desktop.py, agent.py, config.py, runtime_release.py, pairing_client.py, state_store.py, task_executor.py, log_safety.py and relevant tests. Read README.md, docs/WORKER_ONBOARDING.md, docs/OFFLINE_RUNTIME.md, docs/STEP5_LAPTOP_TEST_AND_STEP6_GUIDE.md and output/onboarding/step5-* records. Earlier exported PDFs are dated snapshots.

Contracts that must stay true

Only an approved identity starts a job agent. Preflight/import runs tiny probes without enrolling a node. No shared admin/enrollment/storage token enters the worker. Docker image IDs, owner CPU/RAM and zero extra swap remain enforced. Pause drains; Stop cancels; stale attempts cannot accept output. Exact image identities and once-only accepted accounting remain backend requirements.

03 / Task 6.1: frozen entry points

Lead and deliverable

Vaibhav leads; Kushagra reviews execution/auth compatibility. Produce a working frozen GUI plus a defined bundled agent/preflight helper before building installer UX. Refactor the agent entry section into a callable entry point while retaining current source CLI behavior. New entry-point file names are implementation choices; none is supplied by this documentation.

Recommended two-executable contract

A windowed GUI executable opens the owner interface. A separate console-capable agent executable handles normal agent and --preflight modes; the GUI launches it with hidden-console process flags. This keeps preflight JSON/log output available through redirected pipes without needing a GUI executable to impersonate Python. A single executable with explicit mode dispatch is an alternative, but requires careful early routing and console-output handling.

Centralize the command selection

Introduce one small command builder that returns source commands in development and installed helper commands when frozen. Preflight and normal Start must both use it. Resolve helpers from the trusted installed bundle, not the current working directory or an arbitrary PATH entry. Do not run a Python script by passing its path to a frozen GUI executable; do not open another GUI each time a task is requested.

Assets and storage

Include Tk/Tcl runtime, required modules/native dependencies, app resources and releases/current.json through a reproducible spec/build script. Match bundled resource paths to module-relative lookup [1]. Keep writable state/logs/task staging outside the install directory; do not write into a temporary bundle extraction directory. Never package .env, copied virtual environments, credentials, DPAPI state or coordinator data.

Frozen-process pitfalls to check

A windowed Windows app may lack stdout/stderr, so GUI logging and preflight reporting must tolerate that [2]. Frozen library lookup may interfere with external programs; verify Docker CLI launch against the same SDK endpoint and handle platform-specific environment/library behavior [2]. Test helper spawn, cancellation, crash and app close with the actual binaries, not only mocked sys.frozen flags.

Acceptance for this substep

From a non-repository working directory, the frozen app loads resources, finds its own agent, fails clearly when Docker is absent, and runs pinned real preflight plus an approved image job. No source interpreter/path dependency remains. Capture executable/version/hash and precise failures. Do not proceed to installation polish while the frozen computation path is broken.

04 / Task 6.2: installer and build

Owner and compiler decision

Vaibhav owns the spec/build pipeline; Chinmay owns install text/shortcuts. Start with a reproducible Windows amd64 one-folder app build. NSIS and Inno Setup are installer candidates; neither is installed or configured by this handoff. Review the compiler's license/support terms, including automated-build usage [3, 4], before choosing and pinning a tested version.

Per-user installation

Prefer the current user's dedicated Programs/app directory with ordinary user privileges. Create a Start Menu shortcut and optional desktop shortcut, plus a normal uninstall entry. If using Inno Setup, its lowest privilege setting supports non-administrative installs when the payload works without elevation [5]. Keep Docker/WSL prerequisite guidance separate from worker installation; do not elevate the agent to simplify Docker access.

Build inputs and outputs

Commit a reviewed spec, installer script, version metadata and build dependency constraints. Record exact Python/packager/compiler/worker-dependency versions and the trusted workload release. Suggested new paths: packaging/windows/, scripts/Build-WindowsWorker.ps1, and output/installers/. These are proposed deliverables, not files that currently exist. Build in a fresh environment with explicit absolute source/output paths.

Release contents

Bundle only what the owner app/agent needs: compiled GUI and helper, Python/Tk/dependencies, resources, current.json and notices. Do not include API/PostgreSQL/frontend services or secret state. Keep this release's images.tar as a separately verified offline artifact; do not create a second unversioned model/image copy merely to make an installer look complete.

Version and install identity

Use one stable application identity across versions so upgrade/uninstall targets the same app. Show product/version/publisher accurately; use a truthful team publisher name. Separate read-only binaries from retained per-user data. Preserve settings/identity across compatible upgrades, and check state/version compatibility before claiming rollback. Generate artifact SHA-256 and a dependency/license inventory.

Build and publication boundaries

A successful compiler exit is only a build result. Verify installed behavior on a clean supported Windows account/device. Keep unsigned development builds explicitly labeled; do not disable antivirus or misrepresent signing. Code signing, automatic updates, registry/public release hosting and installer distribution are later authorized release actions. Building this project must not install tools globally on a teammate's computer without their approval.

05 / Task 6.3: tray and login startup

Owner and UI behavior

Chinmay leads with Vaibhav on lifecycle. Keep the current safe default: window X stops owned work and exits. Add a clearly labeled explicit Minimize to tray action if background participation is needed; it must tell the owner computing may continue. Do not silently change Close to hide the app. If tray initialization fails, keep a usable visible window.

Tray commands

Provide Open window, Start, Stop, Pause/Resume and Exit. Display status from the existing controller/server truth: ready, running, paused/draining, disconnected/revoked or attention needed. Disable invalid actions during operations. Choose a suitable small tray library only after inspecting dependencies/support. The tray is a control surface; it does not compute, enroll or maintain a parallel task scheduler.

Thread and shutdown rules

Tray callbacks may run outside Tk's UI thread. Route allowed actions through the existing thread-safe event queue and execute UI/controller changes on the Tk thread. Keep network/preflight/installer operations off that thread. Exit cancels preparation, stops the owned agent, waits for bounded background operations, releases locks and removes the tray icon. Test cancellation while import/launch/reporting is pending.

Startup default: off

Expose a persistent, explicit Start app when I sign in choice, disabled on fresh install. The simplest first version opens the app at login and requires explicit Start for computation. Login startup is distinct from approval to execute jobs and from automatic agent start. Do not add auto-compute until separately specified and tested; any later option must require an existing verified approval and respect saved pause/revocation.

Register and revoke only owned startup

Choose one per-user startup mechanism and one application-owned entry. Quote the absolute installed executable path correctly, including spaces. Support enable/disable from the app, update the path during upgrade and remove only this app's entry on uninstall. Reflect whether registration actually succeeded; no silent system-wide scheduled task or Windows service. Test a real sign-out/sign-in or reboot, not only registry text.

Duplicate launch and locks

Retain the existing per-grid desktop/agent locks. Add a packaged desktop-instance guard and a defined bring-to-front behavior if needed; a duplicate shortcut/login launch must not create another agent for the same identity. Do not use global process-name kills as an instance-management strategy. Record whether multiple different grids are supported or intentionally prevented by the product.

06 / Task 6.4: upgrade and uninstall

Owner and stop handshake

Vaibhav leads; Kushagra reviews failure/lease handling and Chinmay communicates the state choices. Before replacing/removing files, request graceful shutdown of this app's GUI/agent and wait for completion. Design a narrowly scoped same-user shutdown channel or a reliable close-before-install flow. Such a channel is not implemented today; document/validate its ownership and acknowledgement rather than inventing a working --shutdown switch in instructions.

Bounded, owned cleanup

Use the existing worker-specific stop request and container labels/state-directory ownership. Cancel active attempts through the existing agent; unfinished work can be retried under lease rules. Stop only processes whose identity/path/session ownership is established. Never blanket-kill Python, Docker or all ewaste-like process names. Do not prune shared Docker images, stop unrelated containers or erase the coordinator database.

Retained user state

Default uninstall policy should explicitly explain whether protected identity/settings/logs are kept. A sensible first version keeps state for same-account reinstall, with a separate deliberate owner-controlled removal option. Keep state under its existing per-grid user directory. Do not recursively delete broad AppData trees. Uninstall itself is not proof that the server identity has been revoked; explain disconnect/revoke separately.

Upgrade identity and pause

A compatible upgrade/reinstall on the same Windows account must preserve node ID, approved limits and saved pause without another pairing. Startup registration must target the new verified app path and preserve the owner's enabled/disabled choice. If old state cannot be read, preserve it and provide recovery guidance; do not clear credentials to force a new enrollment.

Active work and rollback

Test upgrade/uninstall during download, inference, result upload and a paused worker. No cancelled attempt should appear as accepted completion; retries must remain bounded. If graceful shutdown fails, show an actionable refusal or a narrowly owned fallback whose behavior is documented. For rollback, keep a compatible previous installer/source/image release and verify state/schema compatibility; do not assume every downgrade is safe.

Artifacts and observability

Produce sanitized install/uninstall/upgrade diagnostics with no tokens or private state. Log version and operation outcome, not plaintext secrets. Keep a list of exactly which shortcuts, startup entries, install files and state directories the app owns. The acceptance record must show an unrelated process/container survives. Installer success without verified cleanup is incomplete.

07 / Acceptance matrix and team split

Build and clean-machine gate

Vaibhav: fresh Windows amd64 build with pinned toolchain; run from a path containing spaces and outside the repo. Then install/run on a supported machine without system Python/source. Test missing/stopped Docker, wrong container mode, missing/tampered manifest, incomplete archive, low disk and real preflight. A prepared developer machine cannot substitute for this evidence.

Consent and computation gate

Kushagra + Nikhil: new install performs no job work before owner approval. After invitation/approval, ten tracked photos complete as two tasks with exact output/owner/metric accounting. Check actual immutable image IDs and Docker quotas. Saved approval reconnects; pause persists; revoked nodes cannot resume/re-enroll. Preserve the separate-laptop Step 5 evidence instead of silently replacing it with local tests.

Desktop/tray/startup gate

Chinmay + Vaibhav: responsive UI during slow preparation/network failure; explicit minimize shows tray, Open restores window, Pause/Resume reflects server state, Exit stops owned work. Duplicate launches create no duplicate agent. Startup off creates no login launch; startup on opens the documented app behavior; disabling removes the owned entry. Record a real login/reboot test and unchanged pause.

Install lifecycle gate

Vaibhav + Kushagra: same-account upgrade/reinstall preserves node/limits/pause; protected state copied to another account/machine fails with guidance; uninstall stops only owned activity and removes owned startup/shortcuts. Validate the declared state-retention choice and an unrelated container's survival. Test active-operation close/uninstall; retain failure/retry and cleanup evidence.

Regression and evidence gate

Run meaningful affected worker tests, actual frozen-binary smoke tests and existing backend integration with TEST_DATABASE_URL and TEST_PYTHON when contracts/lifecycle change. Report skipped tests honestly. Current baseline is 84 worker / 33 backend / 32 workload tests; counts are historical checks, not fixed target quotas. Hosted CI, installer testing and actual OS behavior are separate evidence levels.

Record for release review

Suggested outputs: output/onboarding/step6-acceptance.md and step6-verification.json, versioned install artifacts/checksums, build/run instructions, clean-machine screenshots/logs with secrets removed, dependency inventory and current known limits. Record reviewed source commit, toolchain/OS/Docker versions, image release, actual devices, commands, pass/fail/skip and cleanup. This handoff creates none of those implementation artifacts.

08 / Copyable agent brief and references

First receiving-team action

Get the full current source including Step 5, not just the old main commit. Read the runbook/ledgers and any applicable AGENTS.md before editing. Record whether the separate-laptop job gate is closed, user-reported or still pending. Agree on the package/compiler decision and division of work. Commit small reviewed changes; do not overwrite another teammate's state or reset shared databases.

Prompt to give your development agent

Help our team implement Step 6 of E-Waste Compute Grid. Inspect the current repository, docs/OFFLINE_RUNTIME.md, Step 5 verification and this handoff before edits. Harry means teach/architecture; Sejal means implementation/debugging. Steps 1-4 passed local acceptance; Step 5 has local real Docker/API evidence and user-reported separate-laptop preflight, with the final physical job gate still pending in this snapshot. Do not claim an installer already exists.

Implementation scope for that prompt

Build a maintainable Windows amd64 packaged GUI/agent, trusted-resource paths, per-user installer, explicit tray/exit behavior and opt-in login startup. Fix frozen launch/output contracts first. Preserve owner consent, DPAPI identity, grid binding, limits, pause, scoped cancellation/cleanup, release image IDs, leases and exact results. Prefer a simple one-folder build; compare installer tools/license requirements. Do not rewrite the scheduler or introduce a service merely for packaging.

Validation and authority for that prompt

Run affected regressions and actual frozen/installed/clean-machine tests; distinguish mocks, user reports and observed behavior. Produce versioned artifacts/checksums and a dated sanitized Step 6 ledger. No global installation, desktop access, startup change on a real owner's PC, publication, AWS deployment or messages to others is authorized merely by this PDF. Follow the receiving user's explicit scope and explain any necessary permission request.

Official technical references (checked 10 Oct 2026)

[1] PyInstaller runtime paths and executable behavior: https://pyinstaller.org/en/stable/runtime-information.html
[2] PyInstaller frozen subprocess/library and windowed I/O pitfalls: https://pyinstaller.org/en/stable/common-issues-and-pitfalls.html
[3] NSIS license information: https://nsis.sourceforge.io/License
[4] Inno Setup current commercial/build licensing: https://jrsoftware.org/isorder.php
[5] Inno Setup non-administrative install setting: https://jrsoftware.org/ishelp/topic_setup_privilegesrequired.htm

Rebuild and maintain this context

Editable companion: docs/STEP6_WINDOWS_INSTALLER_TEAM_HANDOFF.md. Generator: scripts/Build-Step6TeamHandoff.py, using ReportLab and pypdf. Update dated evidence/status after milestones, render every exported PDF page and inspect layout. This documentation is a proposed implementation plan; actual source and new verified results take precedence.